

BZFlag 2.4 Client Rebuild - Final Report

A ground-up modernization of the graphics pipeline of the BZFlag 2.4 client, built and tested for personal use on ARM64 Linux (Adreno X1-85 GPU, Mesa 26, OpenGL 4.6). Done in three tiers, each delivered as an installable package and tested in live play before the next was started.

Headline results

Metric	Before rebuild	After rebuild
Frame rate (FPS)	15-20	120+
Input lag	~15 ms	1-2 ms
Rendering method	Legacy per-vertex calls (240 glBegin sites)	Batched triangle drawing (glDrawArrays), VBO + shader tank path
Performance gain	-	~600-800% higher frame rate, ~90% lower input lag

What changed

Tier 1 - Tank rendering: the old code submitted every tank vertex one at a time. Tanks now draw from vertex buffer objects (VBOs - geometry pre-loaded onto the GPU) with a purpose-built lighting shader, including proper team colours, shadows and per-tank lighting.

Tier 2 - Lighting and image quality: shader-based lighting for tanks and world geometry, with anti-aliasing and texture filtering upgrades.

Tier 3 - Batching the rest: 97 of the remaining 100 legacy draw sites across 23 source files were converted to batched triangle drawing (GLBatch: geometry is accumulated in memory and submitted as one GPU call instead of hundreds of tiny driver calls). The remaining sites live inside display lists (fonts, sky, weather, flag wave) where batching is not technically possible; they are untouched and correct.

Windowing/input: SDL 1.2 window and input layer upgraded to SDL 2 (SDL 2.32), which is maintained, supports modern display servers and allows vertical-sync and resolution control.

Protocol and gameplay are unchanged: server protocol (0221) untouched, no new gameplay behaviour, no new libraries required at runtime.

Why it is faster

The original 2003-era renderer issued one OpenGL call per vertex (immediate mode). On a modern GPU and driver, that call overhead dominates: the CPU spends its time translating calls instead of the GPU drawing triangles. The rebuild moves all bulk geometry to batched triangle draws with GPU-resident buffers, cutting per-frame OpenGL calls by roughly two orders of magnitude. Frame time on the test machine went from 50-65 ms to under 8 ms.

Testing

Each tier was played and verified live (local solo-bot server and public server) before being accepted: HUD, radar, menus, tanks, shots, flags, weather, ghost boxes and wall geometry all checked visually at each step. Three regressions were caught and fixed during testing (a missing-texture state leak, a flush-overhead lag spike, and tank textures dropping until death); the final build shows none of them.

Deliverables

- bzflag_2.4.31-17_arm64.deb - final package (earlier stage packages -11 through -16 retained for rollback)
- Source: github.com/pc-support-services/bzflag-arm64-sp, branch 2.4, one commit per stage
- Technical detail: bzflag-renderer-tier3-report.pdf (per-stage conversion table)